



ADITYA UNIVERSITY

SCHOOL OF COMPUTING

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Cornerstone project work entitled “**Software Management System**” is submitted by

G. KUSHAL (25B11CS283)

S. ABHISHEK (25B11CS909)

M. SHANKAR (25B11CS598)

S.K ZHUWAO (25B11CSA75)

in partial fulfillment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering for the academic year 2026-2027.

Project Guide

Mrs. K Vydehi, M.Tech.,(Ph.D),
Assistant Professor,
Department of CSE

Head of the Department

Dr. Phani Sridhar Addepalli, Ph.D,
Assistant Professor,
Department of CSE

DECLARATION

We hereby declare that the Cornerstone project entitled “**Software Management System**” is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. Degree in Computer Science and Engineering**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy.

Place: Surampalem

Date: 02-10-2026

By

G. KUSHAL (25B11CS283)

S. ABHISHEK (25B11CS909)

M. SHANKAR (25B11CS598)

S.K ZHUWAO (25B11CSA75)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **K Vydehi, Assistant Professor**, who has guided us throughout the project. Their valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. Chandra Sekhar Kolli, Associate Professor and Deputy HOD-1, Dr. Hari Jyothula, Associate Professor and Deputy HOD-2, and Dr. Pennada Siva Satya Prasad, Assistant Professor and Deputy HOD-3**, Department of Computer Science and Engineering, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. Phani Sridhar Addepalli, Assistant Professor and HOD**, for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing**, for his encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University**, for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

TABLE OF CONTENTS

S.No	Topic	Page No
1	Title Page	1
2	Certificate	2
3	Declaration	3
4	Acknowledgement	4
5	Table of Contents	5
6	Abstract	6
7	List of Figures	7
8	List of Tables	8
9	Acronyms	9
10	1. Introduction	10
11	1.1 Brief Information about the Project	10
12	1.2 Motivation and Contribution of Project	10
13	1.3 Objectives of the Project	10
14	1.4 Scope of the Project	10
15	2. System Analysis	11
16	2.1 Existing System	11
17	2.2 Proposed System	11
18	2.3 Functional Requirements	11
19	2.4 Non-Functional Requirements	11
20	2.5 End Users	12
21	2.6 Requirements Specification	12
22	3. Database Design	13
23	3.1 Major Entities	13
24	3.2 ER Model	13
25	3.3 Relationships and Cardinalities	13
26	4. ER Diagram	14
27	5. Relational Schema	15
28	6. Table Design	16
29	7. Normalization	19
30	8. SQL Implementation	24
31	8.1 DDL – Data Definition Language	24
32	8.2 DML – Data Manipulation Language	26
33	8.3 DQL – Data Query Language	26
34	9. SQL Queries & Outputs	27
35	10. Conclusion	37
36	11. Future Enhancements	37
37	12. References	37

ABSTRACT

The Software Management System is a relational database application designed to manage the information involved in software development work in an organization. It centrally stores employees, managers, projects, tasks, software modules, and clients. The system replaces manual and scattered record keeping with a single organized database in which employee-to-manager reporting, employee-to-project participation, task assignment, task dependencies, project modules, and client-module relationships are represented through primary keys, foreign keys, and relationship tables. NOT NULL, UNIQUE, and CHECK constraints preserve data integrity. The database is normalized up to Third Normal Form (3NF) to reduce redundancy and update anomalies. The design is implemented in Oracle SQL, and fifteen SQL queries demonstrate selection, filtering, sorting, aggregation, grouping, and join operations.

LIST OF FIGURES

Fig No	Title	Page No
1	Displaying the ER diagram of Software Management System	14
2	Displaying the relational schema of Software Management System	15
3	Displaying all employee details	27
4	Displaying employee names and cities	28
5	Displaying employees from Hyderabad	28
6	Displaying high-priority tasks	29
7	Displaying distinct employee cities in alphabetical order	29
8	Displaying projects in alphabetical order	30
9	Displaying the total number of employees	30
10	Displaying employees with their managers	31
11	Displaying projects with their modules and versions	32
12	Displaying employee names, manager names, and cities	33
13	Displaying cities having more than one employee	34
14	Displaying employees hired after January 1, 2023	34
15	Displaying task priorities having more than two tasks	35
16	Displaying module versions and their counts	35
17	Displaying project count by technology	36

LIST OF TABLES

S.No	Topic	Page No
1	Functional Requirements	11
2	End Users and Responsibilities	12
3	Requirements Specification	12
4	Major Database Entities	13
5	Relationships and Cardinalities	13
6	Relational Schema Summary	15
7	Manager Table Structure	16
8	Employee Table Structure	16
9	Project Table Structure	16
10	Task Table Structure	17
11	Module Table Structure	17
12	Client Table Structure	17
13	Works_On Table Structure	17
14	Assigned_To Table Structure	18
15	Task_Dependency Table Structure	18
16	Keys and Constraints	18
17	PROJECT Relation in 1NF	20
18	PROJECT Relation in 2NF	20
19	PROJECT_TECHNOLOGY Relation	21
20	PROJECT Relation in 3NF	22
21	PROJECT_TECHNOLOGY Relation in 3NF	22
22	SQL Query Categories	26

ACRONYMS

DBMS – Database Management System

ER – Entity Relationship

SQL – Structured Query Language

DDL – Data Definition Language

DML – Data Manipulation Language

DQL – Data Query Language

PK – Primary Key

FK – Foreign Key

1NF – First Normal Form

2NF – Second Normal Form

3NF – Third Normal Form

CRUD – Create, Read, Update, Delete

CSE – Computer Science and Engineering

1. INTRODUCTION

1.1 Brief Information about the Project

The Software Management System is a database application that organizes the people, projects, tasks, modules, and clients involved in software development. Managing this information manually is time-consuming and error-prone, and it becomes difficult to search, update, and keep consistent as the number of records grows. The project provides a centralized relational database that stores all of this information in one place and supports quick, accurate retrieval. The system is implemented using Oracle SQL with proper primary keys, foreign keys, and integrity constraints.

1.2 Motivation and Contribution of Project

The project is motivated by the problems of handling software-related records manually: errors in data entry, duplicated information, difficulty in searching and updating records, and the lack of a single trustworthy source of information. A centralized database provides a systematic way to store related data and to enforce its relationships through keys and constraints.

- Centralizing employee, manager, project, task, module, and client records in one relational database.
- Reducing manual work, data duplication, and human errors.
- Maintaining referential integrity using primary and foreign keys.
- Providing faster access to software information through SQL queries.
- Applying normalization (1NF, 2NF, 3NF) to obtain a clean and consistent design.

1.3 Objectives of the Project

- To store and manage software, employee, and project details securely.
- To maintain complete information about projects and their modules, including versions and features.
- To manage employees, their managers, and their project and task assignments.
- To record task priorities and dependencies between tasks.
- To maintain client information and the modules each client maintains.
- To reduce manual work and data-entry errors.
- To provide quick and organized access to records through SQL queries.

1.4 Scope of the Project

The scope covers database-oriented management of employees, managers, projects, tasks, modules, and clients. The system can serve as the backend of a software management application and can be extended later with software license tracking, vendor and purchase management, installation records, renewal alerts, and a web or mobile user interface.

2. SYSTEM ANALYSIS

2.1 Existing System

- Software, user, and project information is maintained manually.
- Records are scattered across notebooks or separate files.
- Searching and updating information takes a long time.
- Checking the availability and status of software or resources is difficult.
- Duplicate data and human errors are common.
- Software purchases, licenses, installations, and renewals need careful manual record keeping.

2.2 Proposed System

The proposed Software Management System provides a centralized database that manages software projects, modules, employees, managers, tasks, and clients in an organized and efficient manner. Information is divided into related tables connected through primary keys, foreign keys, and relationship tables, and constraints keep the data valid and consistent. The system is needed to:

- Reduce manual work in software management.
- Keep software, project, and employee information in one place.
- Provide faster access to software information.
- Reduce data duplication and human errors.
- Maintain accurate and consistent records.

2.3 Functional Requirements

Module	Functional Requirement
Software / Project Management	Add, update, delete, view, and search software projects and their technologies, start dates, and end dates.
Module Management	Maintain the modules of each project with their version and features.
Employee Management	Register employees with contact and address details, hire date, and reporting manager.
Manager Management	Store manager details and view the employees reporting to each manager.
Task Management	Record tasks with description and priority (High, Medium, Low) and the dependencies between tasks.
Assignment Management	Record which employees work on which projects and which tasks are assigned to them.
Client Management	Store client details and the modules maintained for each client.
Search and Reporting	Retrieve and summarize information using SQL queries.

Table 1. Functional Requirements

2.4 Non-Functional Requirements

- **Security** – protect software, project, and user information from unauthorized access.

- **Performance** – retrieve and update records quickly.
- **Reliability** – maintain accurate and consistent software data.
- **Usability** – provide a simple and user-friendly interface.
- **Scalability** – allow the database to handle more software, users, and projects in the future.
- **Data Integrity** – ensure that stored information remains accurate and consistent.
- **Availability** – make the system available whenever software information is required.

2.5 End Users

End User	Responsibilities
Administrator	Manages the complete system; adds and updates software information; manages users and licenses; monitors purchases and renewals.
IT Staff	Registers software; manages software installations; checks availability; records renewals and updates.
User / Employee	Provides personal details for registration; views available software; requests software access; uses assigned software.

Table 2. End Users and Responsibilities

2.6 Requirements Specification

Category	Suggested Requirement
Hardware	Intel i3 or above; 4 GB RAM minimum; adequate storage; standard display and keyboard/mouse.
Operating System	Windows 10/11 or an equivalent supported platform.
Database	Oracle Database / Oracle SQL environment used for the implementation.

Table 3. Requirements Specification

3. DATABASE DESIGN

3.1 Major Entities

Entity	Primary Key	Description / Main Attributes
EMPLOYEE	Emp_ID	Emp_Name, Email, Phone_No, Hire_Date, Address (Street, City, Pincode)
MANAGER	Manager_ID	Manager_Name, Email, Phone_No
PROJECT	Project_ID	Project_Name, Start_Date, End_Date, Technologies, Budget (derived)
TASK	Task_ID	Task_Name, Description, Priority
MODULE	Module_ID	Module_Name, Version, Feature
CLIENT	Client_ID	Client_Name, Contact_Details (Email, Phone_No)

Table 4. Major Database Entities

3.2 ER Model

The Entity-Relationship (ER) model represents the structure of the Software Management System. It identifies entities (objects involved in the system), attributes (properties that describe each entity), primary keys (which uniquely identify each record), relationships (how entities are connected), and cardinality (how many records of one entity can be associated with another).

In the ER model, EMPLOYEE has a composite Address attribute made of Street, City, and Pincode, and Email and Phone_No are shown as key-type attributes; Phone_No is drawn as a multivalued attribute for EMPLOYEE, MANAGER, and CLIENT; Technologies is a multivalued attribute of PROJECT, and Budget is a derived attribute of PROJECT. CLIENT has a composite Contact_Details attribute made of Email and Phone_No.

3.3 Relationships and Cardinalities

Relationship	Entities Involved	Cardinality
WORKS_ON	Employee – Project	M : N
REPORTS_TO	Employee – Manager	M : 1
ASSIGNED_TO	Employee – Project – Task	Ternary
DEVELOPS	Project – Module	1 : M
DEPENDS_ON	Task – Task	M : N
MAINTAINS	Client – Module	1 : M

Table 5. Relationships and Cardinalities

4. ER DIAGRAM

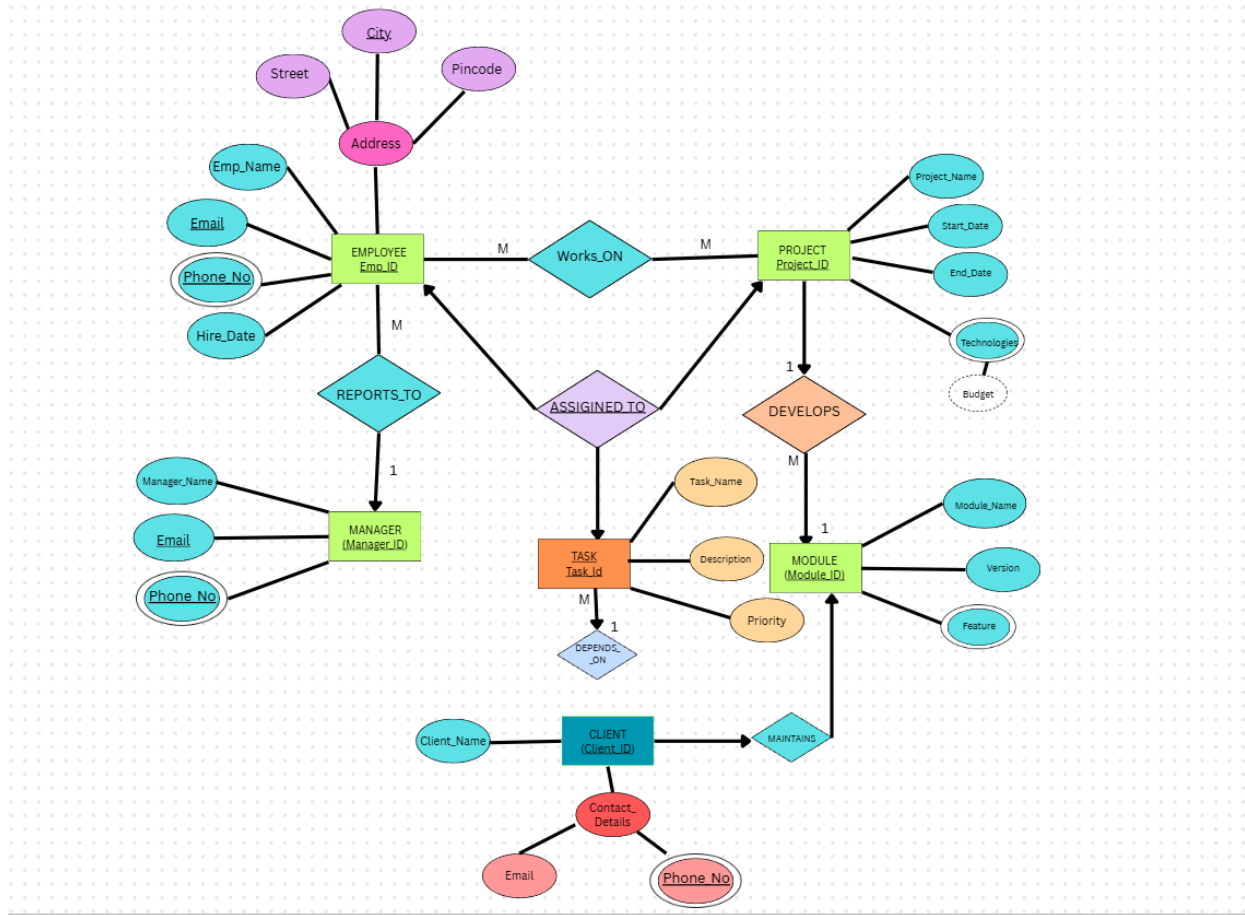


Figure 1 displaying the ER diagram of Software Management System

The ER diagram shows the six entities, their attributes, and the six relationships used in the database. Key attributes are underlined, multivalued attributes are drawn with double ellipses, and the derived attribute Budget is drawn with a dashed ellipse. During relational mapping the M:N relationships and the ternary relationship are converted into separate relations.

5. RELATIONAL SCHEMA

Figure 1 displaying the relational schema of Software Management System

Relation	Relational Schema
MANAGER	Manager_ID (PK), Manager_Name, Email, Phone_No
EMPLOYEE	Emp_ID (PK), Emp_Name, Email, Phone_No, Street, City, Pincode, Hire_Date, Manager_ID (FK)
PROJECT	Project_ID (PK), Project_Name, Start_Date, End_Date, Technologies
TASK	Task_ID (PK), Task_Name, Description, Priority
CLIENT	Client_ID (PK), Client_Name, Email, Phone_No
MODULE	Module_ID (PK), Module_Name, Version, Feature, Project_ID (FK), Client_ID (FK)
WORKS_ON	Emp_ID (PK/FK), Project_ID (PK/FK)
ASSIGNED_TO	Emp_ID (PK/FK), Project_ID (PK/FK), Task_ID (PK/FK)
TASK_DEPENDENCY	Task_ID (PK/FK), Depends_On_Task_ID (PK/FK)

Table 6. Relational Schema Summary

The schema contains nine relations. REPORTS_TO is implemented by the Manager_ID foreign key in EMPLOYEE, DEVELOPS by the Project_ID foreign key in MODULE, and MAINTAINS by the Client_ID foreign key in MODULE. The many-to-many WORKS_ON and DEPENDS_ON relationships and the ternary ASSIGNED_TO relationship are implemented as the separate relations WORKS_ON, TASK_DEPENDENCY, and ASSIGNED_TO.

6. TABLE DESIGN

6.1 Manager Table

Attribute	Data Type	Constraint	Description
Manager_ID	NUMBER(5)	PK	Unique manager identifier
Manager_Name	VARCHAR2(50)	NOT NULL	Manager name
Email	VARCHAR2(100)	UNIQUE, NOT NULL	Manager email
Phone_No	VARCHAR2(15)	UNIQUE, NOT NULL	Manager phone number

Table 7. Manager Table Structure

6.2 Employee Table

Attribute	Data Type	Constraint	Description
Emp_ID	NUMBER(5)	PK	Unique employee identifier
Emp_Name	VARCHAR2(50)	NOT NULL	Employee name
Email	VARCHAR2(100)	UNIQUE, NOT NULL	Employee email
Phone_No	VARCHAR2(15)	UNIQUE, NOT NULL	Employee phone number
Street	VARCHAR2(100)	—	Street address
City	VARCHAR2(50)	—	City
Pincode	VARCHAR2(10)	—	Postal code
Hire_Date	DATE	NOT NULL	Date of joining
Manager_ID	NUMBER(5)	FK	References MANAGER (REPORTS_TO)

Table 8. Employee Table Structure

6.3 Project Table

Attribute	Data Type	Constraint	Description
Project_ID	NUMBER(5)	PK	Unique project identifier
Project_Name	VARCHAR2(100)	NOT NULL	Project name
Start_Date	DATE	NOT NULL	Project start date
End_Date	DATE	CHECK	Project end date ($\geq$ Start_Date or NULL)
Technologies	VARCHAR2(200)	—	Technologies used

Table 9. Project Table Structure

6.4 Task Table

Attribute	Data Type	Constraint	Description
Task_ID	NUMBER(5)	PK	Unique task identifier
Task_Name	VARCHAR2(100)	NOT NULL	Task name
Description	VARCHAR2(255)	—	Task description
Priority	VARCHAR2(20)	CHECK	High, Medium, or Low

Table 10. Task Table Structure

6.5 Module Table

Attribute	Data Type	Constraint	Description
Module_ID	NUMBER(5)	PK	Unique module identifier
Module_Name	VARCHAR2(100)	NOT NULL	Module name
Version	VARCHAR2(20)	NOT NULL	Module version
Feature	VARCHAR2(255)	—	Module feature
Project_ID	NUMBER(5)	FK, NOT NULL	References PROJECT (DEVELOPS)
Client_ID	NUMBER(5)	FK	References CLIENT (MAINTAINS)

Table 11. Module Table Structure

6.6 Client Table

Attribute	Data Type	Constraint	Description
Client_ID	NUMBER(5)	PK	Unique client identifier
Client_Name	VARCHAR2(100)	NOT NULL	Client name
Email	VARCHAR2(100)	UNIQUE, NOT NULL	Client email
Phone_No	VARCHAR2(15)	UNIQUE, NOT NULL	Client phone number

Table 12. Client Table Structure

6.7 Relationship Tables

Attribute	Data Type	Constraint	Description
Emp_ID	NUMBER(5)	PK, FK	References EMPLOYEE
Project_ID	NUMBER(5)	PK, FK	References PROJECT

Table 13. Works_On Table Structure

Attribute	Data Type	Constraint	Description
Emp_ID	NUMBER(5)	PK, FK	References EMPLOYEE
Project_ID	NUMBER(5)	PK, FK	References PROJECT
Task_ID	NUMBER(5)	PK, FK	References TASK

Table 14. Assigned_To Table Structure

Attribute	Data Type	Constraint	Description
Task_ID	NUMBER(5)	PK, FK	References TASK
Depends_On_Task_ID	NUMBER(5)	PK, FK	References TASK (the task it depends on)

Table 15. Task_Dependency Table Structure

6.8 Keys and Constraints

Constraint Type	Applied To / Purpose
Primary Key	Uniquely identifies the rows of every table.
Foreign Key	Connects EMPLOYEE to MANAGER, MODULE to PROJECT and CLIENT, and the relationship tables to their parent tables.
NOT NULL	Required names, dates, versions, e-mail, and phone numbers.
UNIQUE	Email and Phone_No of MANAGER, EMPLOYEE, and CLIENT must not repeat.
CHECK	End_Date IS NULL OR End_Date $\geq$ Start_Date (PROJECT); Priority IN (High, Medium, Low) (TASK).
Composite Key	(Emp_ID, Project_ID), (Emp_ID, Project_ID, Task_ID), and (Task_ID, Depends_On_Task_ID) prevent duplicate relationship records.

Table 16. Keys and Constraints

7. NORMALIZATION

Normalization is a systematic process of organizing data into well-structured tables to reduce data redundancy, avoid data inconsistency, and prevent insertion, deletion, and update anomalies. A large table is divided into smaller related tables, and the relationships between them are established using primary and foreign keys. The commonly used normal forms are 1NF, 2NF, and 3NF. The final database is designed up to Third Normal Form (3NF).

7.1 First Normal Form (1NF)

A relation is in 1NF if each column contains atomic (single) values, there are no repeating groups or multivalued attributes, and each row is uniquely identifiable using a primary key. One cell should contain only one value.

MANAGER is already in 1NF: Manager_ID is the primary key, and name, e-mail, and phone number each hold a single value. In **PROJECT**, some projects use two technologies in one cell (for example “Java, MySQL”). Each technology is therefore written as a separate value, so every column holds a single value and there are no repeating groups.

Project_ID	Project_Name	Start_Date	End_Date	Technology
301	Banking System	01-JAN-24	30-JUN-24	Java
301	Banking System	01-JAN-24	30-JUN-24	MySQL
302	E-Commerce Platform	01-FEB-24	31-AUG-24	Python
302	E-Commerce Platform	01-FEB-24	31-AUG-24	Django
303	Hospital Management	01-MAR-24	30-SEP-24	Java
303	Hospital Management	01-MAR-24	30-SEP-24	MySQL
304	College Portal	01-APR-24	31-OCT-24	PHP
304	College Portal	01-APR-24	31-OCT-24	MySQL
305	Food Delivery App	01-MAY-24	30-NOV-24	Flutter
305	Food Delivery App	01-MAY-24	30-NOV-24	Firebase
306	Online Library	01-JUN-24	31-DEC-24	Python
306	Online Library	01-JUN-24	31-DEC-24	MySQL
307	HR Management	01-JUL-24	31-JAN-25	Java
307	HR Management	01-JUL-24	31-JAN-25	Oracle
308	Inventory System	01-AUG-24	28-FEB-25	C#
308	Inventory System	01-AUG-24	28-FEB-25	SQL Server
309	Travel Booking	01-SEP-24	31-MAR-25	React
309	Travel Booking	01-SEP-24	31-MAR-25	Node.js
310	Online Learning	01-OCT-24	30-APR-25	Python
310	Online Learning	01-OCT-24	30-APR-25	Flask
311	Banking Mobile App	01-NOV-24	31-MAY-25	Android

Project_ID	Project_Name	Start_Date	End_Date	Technology
311	Banking Mobile App	01-NOV-24	31-MAY-25	Kotlin
312	Hospital App	01-DEC-24	30-JUN-25	React Native

Table 17. PROJECT Relation in 1NF

7.2 Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and every non-key attribute is fully functionally dependent on the entire primary key, i.e., there is no partial dependency. After the 1NF step, the key of the table is the composite (Project_ID, Technology). Project_Name, Start_Date, and End_Date depend only on Project_ID, not on the complete composite key, which is a partial dependency. The table is therefore split into PROJECT and PROJECT_TECHNOLOGY, and both are now in 2NF.

Project_ID	Project_Name	Start_Date	End_Date
301	Banking System	01-JAN-24	30-JUN-24
302	E-Commerce Platform	01-FEB-24	31-AUG-24
303	Hospital Management	01-MAR-24	30-SEP-24
304	College Portal	01-APR-24	31-OCT-24
305	Food Delivery App	01-MAY-24	30-NOV-24
306	Online Library	01-JUN-24	31-DEC-24
307	HR Management	01-JUL-24	31-JAN-25
308	Inventory System	01-AUG-24	28-FEB-25
309	Travel Booking	01-SEP-24	31-MAR-25
310	Online Learning	01-OCT-24	30-APR-25
311	Banking Mobile App	01-NOV-24	31-MAY-25
312	Hospital App	01-DEC-24	30-JUN-25

Table 18. PROJECT Relation in 2NF

Project_ID	Technology
301	Java
301	MySQL
302	Python
302	Django
303	Java
303	MySQL
304	PHP
304	MySQL
305	Flutter

Project_ID	Technology
305	Firebase
306	Python
306	MySQL
307	Java
307	Oracle
308	C#
308	SQL Server
309	React
309	Node.js
310	Python
310	Flask
311	Android
311	Kotlin
312	React Native

Table 19. PROJECT_TECHNOLOGY Relation

7.3 Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and has no transitive dependency, i.e., non-key attributes depend directly on the primary key and not on another non-key attribute. A transitive dependency occurs when a non-key attribute depends on another non-key attribute.

The relations obtained after 2NF are checked for transitive dependencies:

- **PROJECT** – Project_ID is the primary key, and Project_Name, Start_Date, and End_Date depend directly on Project_ID.
- **PROJECT_TECHNOLOGY** – the composite primary key is (Project_ID, Technology), and Project_ID is a foreign key that references PROJECT(Project_ID).

In 2NF the partial dependency was removed by separating PROJECT and PROJECT_TECHNOLOGY. There is no transitive dependency because every non-key attribute depends directly on the primary key and not on another non-key attribute. Therefore both relations are in 3NF.

Project_ID (PK)	Project_Name	Start_Date	End_Date
301	Banking System	01-JAN-24	30-JUN-24
302	E-Commerce Platform	01-FEB-24	31-AUG-24
303	Hospital Management	01-MAR-24	30-SEP-24
304	College Portal	01-APR-24	31-OCT-24
305	Food Delivery App	01-MAY-24	30-NOV-24

Project_ID (PK)	Project_Name	Start_Date	End_Date
306	Online Library	01-JUN-24	31-DEC-24
307	HR Management	01-JUL-24	31-JAN-25
308	Inventory System	01-AUG-24	28-FEB-25
309	Travel Booking	01-SEP-24	31-MAR-25
310	Online Learning	01-OCT-24	30-APR-25
311	Banking Mobile App	01-NOV-24	31-MAY-25
312	Hospital App	01-DEC-24	30-JUN-25

Table 20. PROJECT Relation in 3NF

Project_ID (PK, FK)	Technology (PK)
301	Java
301	MySQL
302	Python
302	Django
303	Java
303	MySQL
304	PHP
304	MySQL
305	Flutter
305	Firebase
306	Python
306	MySQL
307	Java
307	Oracle
308	C#
308	SQL Server
309	React
309	Node.js
310	Python
310	Flask
311	Android
311	Kotlin
312	React Native

Table 21. PROJECT_TECHNOLOGY Relation in 3NF

Note: the queries in Section 9 were executed on the PROJECT table in which the technologies of a project are stored together in the Technologies column (for example, Q15).

8. SQL IMPLEMENTATION

The following Oracle SQL implementation creates the relational tables obtained from the ER-to-relational mapping. The schema uses primary keys, foreign keys, NOT NULL, UNIQUE, and CHECK constraints.

8.1 DDL – Data Definition Language

```
CREATE TABLE MANAGER (  
    Manager_ID  NUMBER(5) PRIMARY KEY,  
    Manager_Name VARCHAR2(50) NOT NULL,  
    Email       VARCHAR2(100) UNIQUE NOT NULL,  
    Phone_No    VARCHAR2(15) UNIQUE NOT NULL  
);  
  
CREATE TABLE EMPLOYEE (  
    Emp_ID      NUMBER(5) PRIMARY KEY,  
    Emp_Name    VARCHAR2(50) NOT NULL,  
    Email       VARCHAR2(100) UNIQUE NOT NULL,  
    Phone_No    VARCHAR2(15) UNIQUE NOT NULL,  
    Street      VARCHAR2(100),  
    City        VARCHAR2(50),  
    Pincode     VARCHAR2(10),  
    Hire_Date   DATE NOT NULL,  
    Manager_ID  NUMBER(5),  
    CONSTRAINT fk_emp_manager  
        FOREIGN KEY (Manager_ID)  
        REFERENCES MANAGER(Manager_ID)  
);  
  
CREATE TABLE PROJECT (  
    Project_ID  NUMBER(5) PRIMARY KEY,  
    Project_Name VARCHAR2(100) NOT NULL,  
    Start_Date  DATE NOT NULL,  
    End_Date    DATE,  
    Technologies VARCHAR2(200),  
    CONSTRAINT chk_project_dates  
        CHECK (End_Date IS NULL OR End_Date >= Start_Date)  
);  
  
CREATE TABLE TASK (  
    Task_ID     NUMBER(5) PRIMARY KEY,  
    Task_Name   VARCHAR2(100) NOT NULL,  
    Description  VARCHAR2(255),  
    Priority     VARCHAR2(20),  
    CONSTRAINT chk_task_priority  
        CHECK (Priority IN ('High', 'Medium', 'Low'))  
);  
  
CREATE TABLE CLIENT (  

```

```

Client_ID NUMBER(5) PRIMARY KEY,
Client_Name VARCHAR2(100) NOT NULL,
Email VARCHAR2(100) UNIQUE NOT NULL,
Phone_No VARCHAR2(15) UNIQUE NOT NULL
);

CREATE TABLE MODULE (
Module_ID NUMBER(5) PRIMARY KEY,
Module_Name VARCHAR2(100) NOT NULL,
Version VARCHAR2(20) NOT NULL,
Feature VARCHAR2(255),
Project_ID NUMBER(5) NOT NULL,
Client_ID NUMBER(5),
CONSTRAINT fk_module_project
FOREIGN KEY (Project_ID) REFERENCES PROJECT(Project_ID),
CONSTRAINT fk_module_client
FOREIGN KEY (Client_ID) REFERENCES CLIENT(Client_ID)
);

CREATE TABLE WORKS_ON (
Emp_ID NUMBER(5),
Project_ID NUMBER(5),
PRIMARY KEY (Emp_ID, Project_ID),
FOREIGN KEY (Emp_ID) REFERENCES EMPLOYEE(Emp_ID),
FOREIGN KEY (Project_ID) REFERENCES PROJECT(Project_ID)
);

CREATE TABLE ASSIGNED_TO (
Emp_ID NUMBER(5),
Project_ID NUMBER(5),
Task_ID NUMBER(5),
PRIMARY KEY (Emp_ID, Project_ID, Task_ID),
FOREIGN KEY (Emp_ID) REFERENCES EMPLOYEE(Emp_ID),
FOREIGN KEY (Project_ID) REFERENCES PROJECT(Project_ID),
FOREIGN KEY (Task_ID) REFERENCES TASK(Task_ID)
);

CREATE TABLE TASK_DEPENDENCY (
Task_ID NUMBER(5),
Depends_On_Task_ID NUMBER(5),
PRIMARY KEY (Task_ID, Depends_On_Task_ID),
FOREIGN KEY (Task_ID) REFERENCES TASK(Task_ID),
FOREIGN KEY (Depends_On_Task_ID) REFERENCES TASK(Task_ID)
);

```


8.2 DML – Data Manipulation Language

Sample records were inserted into the tables using INSERT statements so that the queries could be tested.
Example inserts:

```
INSERT INTO MANAGER VALUES (201, 'Ramesh Kumar', 'ramesh@company.com',  
'90000000001');  
  
INSERT INTO EMPLOYEE VALUES (101, 'Rahul Kumar', 'rahul@gmail.com', '9876500001',  
'MG Road', 'Hyderabad', '500001', DATE '2022-01-10', 201);  
  
INSERT INTO PROJECT VALUES (301, 'Banking System', DATE '2024-01-01',  
DATE '2024-06-30', 'Java, MySQL');  
  
COMMIT;
```

8.3 DQL – Data Query Language

The queries in Section 9 cover the following categories of data retrieval.

Category	Queries
Selection and projection	Q1, Q2
Filtering (WHERE)	Q3, Q4, Q12
DISTINCT and ORDER BY	Q5, Q6
Aggregate functions	Q7
Joins	Q8, Q9, Q10
GROUP BY and HAVING	Q11, Q13, Q14, Q15

Table 22. SQL Query Categories

9. SQL QUERIES & OUTPUTS

The following SQL queries demonstrate selection, filtering, sorting, joins, aggregation, and grouping on the Software Management System database, together with their outputs.

Q1. Display all employees

```
SELECT * FROM EMPLOYEE;
```

Output: Shows all employee records.

EMP_ID	EMP_NAME	EMAIL	PHONE_NO	STREET	CITY	PINCODE	HIRE_DATE	MANAGER_ID
101	Rahul Kumar	rahul@gmail.com	9876500001	MG Road	Hyderabad	500001	10-JAN-22	201
102	Priya Sharma	priya@gmail.com	9876500002	Main Road	Vijayawada	520001	15-MAR-22	202
103	Arjun Reddy	arjun@gmail.com	9876500003	Park Street	Chennai	600001	20-MAY-22	203
104	Sneha Rao	sneha@gmail.com	9876500004	Lake Road	Bangalore	560001	12-JUL-22	204
105	Kiran Kumar	kiran@gmail.com	9876500005	Gandhi Road	Hyderabad	500002	18-SEP-22	205
106	Anjali Singh	anjali@gmail.com	9876500006	Temple Road	Delhi	110001	05-JAN-23	206
107	Vikram Das	vikram@gmail.com	9876500007	Station Road	Mumbai	400001	14-FEB-23	207
108	Neha Patel	neha@gmail.com	9876500008	Market Road	Ahmedabad	380001	22-APR-23	208
109	Ravi Teja	raviteja@gmail.com	9876500009	College Road	Vizag	530001	10-JUN-23	209
110	Divya Reddy	divya@gmail.com	9876500010	Church Road	Hyderabad	500003	17-AUG-23	210
111	Suresh Babu	suresh@gmail.com	9876500011	Nehru Street	Pune	411001	25-SEP-23	211
112	Pooja Verma	pooja@gmail.com	9876500012	Garden Road	Delhi	110002	01-NOV-23	212
113	Manoj Kumar	manoj@gmail.com	9876500013	Ring Road	Bangalore	560002	11-JAN-24	213
114	Lakshmi Devi	lakshmi@gmail.com	9876500014	Hill Road	Chennai	600002	19-MAR-24	214
115	Akhil Raj	akhil@gmail.com	9876500015	Beach Road	Vizag	530002	28-MAY-24	215

Figure 3 displaying all employee details

Q2. Display employee names and cities

```
SELECT Emp_Name, City  
FROM EMPLOYEE;
```

Output: Shows each employee with the city.

EMP_NAME	CITY
Rahul Kumar	Hyderabad
Priya Sharma	Vijayawada
Arjun Reddy	Chennai
Sneha Rao	Bangalore
Kiran Kumar	Hyderabad
Anjali Singh	Delhi
Vikram Das	Mumbai
Neha Patel	Ahmedabad
Ravi Teja	Vizag
Divya Reddy	Hyderabad
Suresh Babu	Pune
Pooja Verma	Delhi
Manoj Kumar	Bangalore
Lakshmi Devi	Chennai
Akhil Raj	Vizag

Figure 4 displaying employee names and cities

Q3. Find employees from Hyderabad

```
SELECT Emp_ID, Emp_Name, City  
FROM EMPLOYEE  
WHERE City = 'Hyderabad';
```

Output: Filters employees whose city is Hyderabad.

EMP_ID	EMP_NAME	CITY
101	Rahul Kumar	Hyderabad
105	Kiran Kumar	Hyderabad
110	Divya Reddy	Hyderabad

Figure 5 displaying employees from Hyderabad

Q4. Display high-priority tasks

```
SELECT Task_ID, Task_Name, Priority
FROM TASK
WHERE Priority = 'High';
```

Output: Shows the tasks whose priority is High.

TASK_ID	TASK_NAME	PRIORITY
401	Database Design	High
403	API Development	High
404	Login Module	High
406	Bug Fixing	High
409	Payment Integration	High
410	Security Testing	High
412	Deployment	High

Figure 6 displaying high-priority tasks

Q5. Display distinct employee cities

```
SELECT DISTINCT City
FROM EMPLOYEE
ORDER BY City;
```

Output: Lists each city once, in alphabetical order.

CITY
Ahmedabad
Bangalore
Chennai
Delhi
Hyderabad
Mumbai
Pune
Vijayawada
Vizag

Figure 7 displaying distinct employee cities in alphabetical order

Q6. Display projects alphabetically

```
SELECT Project_ID, Project_Name  
FROM PROJECT  
ORDER BY Project_Name;
```

Output: Sorts the projects by name in ascending order.

PROJECT_ID	PROJECT_NAME
311	Banking Mobile App
301	Banking System
313	CRM System
304	College Portal
302	E-Commerce Platform
305	Food Delivery App
307	HR Management
312	Hospital App
303	Hospital Management
308	Inventory System
310	Online Learning
306	Online Library
314	Smart Parking
309	Travel Booking
315	Weather Application

Figure 8 displaying projects in alphabetical order

Q7. Count employees

```
SELECT COUNT(*) AS Total_Employees  
FROM EMPLOYEE;
```

Output: Returns the total number of employees.

TOTAL_EMPLOYEES
15

Figure 9 displaying the total number of employees

Q8. Employees and their managers

```
SELECT E.Emp_ID,  
       E.Emp_Name,  
       M.Manager_Name  
FROM EMPLOYEE E  
JOIN MANAGER M  
ON E.Manager_ID = M.Manager_ID  
ORDER BY E.Emp_ID;
```

Output: Joins EMPLOYEE and MANAGER using Manager_ID.

EMP_ID	EMP_NAME	MANAGER_NAME
101	Rahul Kumar	Ramesh Kumar
102	Priya Sharma	Sanjay Rao
103	Arjun Reddy	Meena Sharma
104	Sneha Rao	Ajay Reddy
105	Kiran Kumar	Kavya Singh
106	Anjali Singh	Mohan Das
107	Vikram Das	Swathi Patel
108	Neha Patel	Vijay Kumar
109	Ravi Teja	Deepak Rao
110	Divya Reddy	Naveen Reddy
111	Suresh Babu	Harish Babu
112	Pooja Verma	Geetha Devi
113	Manoj Kumar	Prakash Kumar
114	Lakshmi Devi	Sunitha Rao
115	Akhil Raj	Rajesh Singh

Figure 10 displaying employees with their managers

Q9. Projects and their modules

```
SELECT P.Project_ID,  
       P.Project_Name,  
       M.Module_Name,  
       M.Version  
FROM PROJECT P  
JOIN MODULE M  
ON P.Project_ID = M.Project_ID  
ORDER BY P.Project_ID;
```

Output: Joins PROJECT and MODULE using Project_ID.

PROJECT_ID	PROJECT_NAME	MODULE_NAME	VERSION
301	Banking System	Authentication	1.0
302	E-Commerce Platform	Payments	1.1
303	Hospital Management	Patient Records	2.0
304	College Portal	Student Portal	1.2
305	Food Delivery App	Order Tracking	1.5
306	Online Library	Catalog	1.0
307	HR Management	Employee Records	2.1
308	Inventory System	Stock Control	1.3
309	Travel Booking	Booking	2.0
310	Online Learning	Course Management	1.4
311	Banking Mobile App	Mobile Banking	2.2
312	Hospital App	Appointments	1.0
313	CRM System	Customer Management	1.1
314	Smart Parking	Parking Control	2.0
315	Weather Application	Forecast	1.5

Figure 11 displaying projects with their modules and versions

Q10. Employees, managers, and cities

```
SELECT E.Emp_Name,  
       M.Manager_Name,  
       E.City  
FROM EMPLOYEE E  
JOIN MANAGER M  
ON E.Manager_ID = M.Manager_ID  
ORDER BY E.Emp_Name;
```

Output: Shows each employee with the manager and city, sorted by employee name.

EMP_NAME	MANAGER_NAME	CITY
Akhil Raj	Rajesh Singh	Vizag
Anjali Singh	Mohan Das	Delhi
Arjun Reddy	Meena Sharma	Chennai
Divya Reddy	Naveen Reddy	Hyderabad
Kiran Kumar	Kavya Singh	Hyderabad
Lakshmi Devi	Sunitha Rao	Chennai
Manoj Kumar	Prakash Kumar	Bangalore
Neha Patel	Vijay Kumar	Ahmedabad
Pooja Verma	Geetha Devi	Delhi
Priya Sharma	Sanjay Rao	Vijayawada
Rahul Kumar	Ramesh Kumar	Hyderabad
Ravi Teja	Deepak Rao	Vizag
Sneha Rao	Ajay Reddy	Bangalore
Suresh Babu	Harish Babu	Pune
Vikram Das	Swathi Patel	Mumbai

Figure 12 displaying employee names, manager names, and cities

Q11. Employees grouped by city

```
SELECT City,  
       COUNT(*) AS Employee_Count  
FROM EMPLOYEE  
GROUP BY City  
HAVING COUNT(*) > 1  
ORDER BY Employee_Count DESC;
```

Output: Shows cities having more than one employee, with the count.

CITY	EMPLOYEE_COUNT
Hyderabad	3
Bangalore	2
Chennai	2
Vizag	2
Delhi	2

Figure 13 displaying cities having more than one employee

Q12. Employees hired after January 1, 2023

```
SELECT Emp_ID,  
       Emp_Name,  
       City,  
       Hire_Date  
FROM EMPLOYEE  
WHERE Hire_Date > DATE '2023-01-01'  
ORDER BY Hire_Date;
```

Output: Lists employees hired after 1 January 2023 in date order.

EMP_ID	EMP_NAME	CITY	HIRE_DATE
106	Anjali Singh	Delhi	05-JAN-23
107	Vikram Das	Mumbai	14-FEB-23
108	Neha Patel	Ahmedabad	22-APR-23
109	Ravi Teja	Vizag	10-JUN-23
110	Divya Reddy	Hyderabad	17-AUG-23
111	Suresh Babu	Pune	25-SEP-23
112	Pooja Verma	Delhi	01-NOV-23
113	Manoj Kumar	Bangalore	11-JAN-24
114	Lakshmi Devi	Chennai	19-MAR-24
115	Akhil Raj	Vizag	28-MAY-24

Figure 14 displaying employees hired after January 1, 2023

Q13. Task priorities having more than 2 tasks

```
SELECT Priority,  
       COUNT(*) AS Task_Count  
FROM TASK  
GROUP BY Priority  
HAVING COUNT(*) > 2  
ORDER BY Task_Count DESC;
```

Output: Shows the priorities that have more than two tasks.

PRIORITY	TASK_COUNT
High	7
Medium	6

Figure 15 displaying task priorities having more than two tasks

Q14. Module versions and their counts

```
SELECT Version,  
       COUNT(*) AS Module_Count  
FROM MODULE  
GROUP BY Version  
ORDER BY Module_Count DESC;
```

Output: Counts the modules for each version.

VERSION	MODULE_COUNT
1.0	3
2.0	3
1.5	2
1.1	2
2.2	1
1.3	1
1.2	1
1.4	1
2.1	1

Figure 16 displaying module versions and their counts

Q15. Project count by technology

```
SELECT Technologies,  
       COUNT(*) AS Project_Count  
FROM PROJECT  
GROUP BY Technologies  
ORDER BY Project_Count DESC;
```

Output: Counts the projects for each technology combination.

TECHNOLOGIES	PROJECT_COUNT
Java, MySQL	2
React, Node.js	1
Flutter, Firebase	1
Android, Kotlin	1
Python, MySQL	1
PHP, MySQL	1
Python, Flask	1
Java, Spring	1
Python, API	1
React Native	1
IoT, Python	1
Python, Django	1
C#, SQL Server	1
Java, Oracle	1

Figure 17 displaying project count by technology

10. CONCLUSION

The Software Management System replaces manual record keeping with a centralized relational database. It keeps employee, manager, project, task, module, and client information in one organized place, which reduces errors and duplicate data. The implementation uses normalized tables with primary and foreign keys, UNIQUE, NOT NULL, and CHECK constraints, and separate relations for many-to-many and ternary relationships. The SQL queries demonstrate retrieval, filtering, sorting, joins, aggregation, and grouping. The system meets its objectives of reducing manual work, giving faster access to records, and keeping data reliable and secure, and it can grow with more users, projects, and records.

11. FUTURE ENHANCEMENTS

- Software license management: license keys, availability, status, and expiry tracking.
- Vendor, purchase, and installation records for software products.
- Automated reminders for license renewals and expiry dates.
- Role-based access for administrators, IT staff, and employees.
- Software access requests and approvals.
- A web or mobile front end for entering and viewing data.
- Dashboards and reports on projects, tasks, and resources.

12. REFERENCES

The following books and online resources were referred to in preparing this documentation.

1. Korth, H. F., Silberschatz, A., and Sudarshan, S., Database System Concepts, McGraw Hill.
2. Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson.
3. Oracle Database SQL Language Reference (Oracle Corporation). Available at: <https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/index.html>
4. W3Schools, SQL Tutorial. Available at: <https://www.w3schools.com/sql/>
5. GeeksforGeeks, DBMS Tutorial. Available at: <https://www.geeksforgeeks.org/dbms/>
6. TutorialsPoint, DBMS Tutorial. Available at: <https://www.tutorialspoint.com/dbms/index.htm>